

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL

CARRERA:

SOFTWARE

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

ASIGNATURA:

APLICACIONES DISTRIBUIDAS

TEMA:

FORM-FB-01 — FICHAS BIBLIOGRÁFICAS — FUNDAMENTOS DE SD Y
ARQUITECTURAS DISTRIBUIDAS

ESTUDIANTE:

ISAIAS ABRAHAM URBINA ROMERO

AÑO LECTIVO

2026 SPA

Ficha Bibliográfica 1

Referencia:

Liu, G., Huang, B., Liang, Z., Qin, M., Zhou, H., & Li, Z. (2020). *Microservices: Architecture, container, and challenges*. Proceedings - Companion of the 2020 IEEE 20th International Conference on Software Quality, Reliability, and Security, QRS-C 2020, 629–635.

Enlace:

<https://doi.org/10.1109/QRS-C51114.2020.00107>

Resumen:

Los microservicios son básicamente una forma de dividir una aplicación grande en partes más pequeñas que funcionan de manera independiente. Surgieron como solución a los problemas que tenían las arquitecturas tradicionales, las cuales se volvían muy difíciles de manejar con el tiempo. Empresas como Netflix y WeChat ya los utilizan masivamente. Para desplegarlos, los contenedores como Docker son la opción más práctica porque son ligeros y fáciles de gestionar. Sin embargo, existen tres problemas importantes que resolver: el rendimiento entre servicios, la complejidad de depurar errores y el mantenimiento de la consistencia de datos entre múltiples bases de datos.

Nota personal:

Esta fuente me ayudó a comprender por qué los microservicios se han vuelto tan utilizados en aplicaciones modernas y cómo los contenedores facilitan su despliegue. También me permitió identificar algunos desafíos importantes, como el rendimiento, la depuración de errores y la consistencia de datos en sistemas distribuidos.

Ficha Bibliográfica 2

Referencia:

Talaver, O. V., & Vakaliuk, T. A. (2023). *Reliable distributed systems: Review of modern approaches*. Journal of Edge Computing, 2(1), 84–101.

Enlace:

<https://doi.org/10.55056/jec.586>

Resumen:

Este artículo analiza cómo lograr que los sistemas distribuidos basados en microservicios funcionen de manera eficiente en entornos reales y no solo en teoría. Los autores explican que los microservicios no siempre representan la mejor opción, sino que son más útiles cuando los equipos de desarrollo crecen y necesitan trabajar de manera independiente. La idea principal es reducir el acoplamiento entre servicios mediante conceptos como *Bounded Context*, el cual ayuda a definir claramente las responsabilidades de cada servicio según el negocio. Además, se describen patrones como Event Sourcing, CQRS y Saga para mejorar la comunicación y el manejo de datos entre servicios.

Nota personal:

Esta fuente me ayudó a entender que los microservicios no siempre son la mejor solución, sino que dependen de las necesidades del sistema y del tamaño de los equipos. Además, comprendí mejor la importancia de reducir el acoplamiento entre servicios y el uso de patrones como CQRS, Event Sourcing y Saga para mejorar la comunicación y organización en sistemas distribuidos.

Ficha Bibliográfica 3

Referencia:

Abad, C. L., Ortiz-Holguín, E., & Boza, E. F. (2021). *Have We Reached Consensus? An Analysis of Distributed Systems Syllabi*. SIGCSE 2021 - Proceedings of the 52nd ACM Technical Symposium on Computer Science Education, 1082–1088.

Enlace:

<https://doi.org/10.1145/3408877.3432409>

Resumen:

Los investigadores de la ESPOL analizaron 51 sílabos de cursos de Sistemas Distribuidos pertenecientes a universidades de diferentes países para identificar los temas más enseñados. Encontraron que existe una gran coincidencia en contenidos como consenso, replicación, tolerancia a fallos y sincronización de tiempo. Los libros más utilizados son los de Van Steen y Coulouris, mientras que muchos de los artículos estudiados provienen de empresas como Google y Amazon. También se observó que el tema de tiempo y orden aparece en el 71 % de los cursos, aunque no figure de manera explícita en las guías oficiales de ACM/IEEE.

Nota personal:

Esta fuente me ayudó a conocer cuáles son los temas más importantes que se enseñan actualmente en Sistemas Distribuidos a nivel internacional. Además, me permitió entender la relevancia de conceptos como consenso, replicación y tolerancia a fallos dentro de la formación académica y profesional.

Ficha Bibliográfica 4

Referencia:

Pinto-Aguero, S., & Noel, R. (2025). *Microservices Evolution Factors: A Multivocal Literature Review*. IEEE Access, 13, 88707–88730.

Enlace:

<https://doi.org/10.1109/ACCESS.2025.3570658>

Resumen:

Este artículo analiza cuáles son los factores que permiten que los microservicios evolucionen adecuadamente con el tiempo y cuáles son los principales obstáculos que enfrentan. Los autores revisaron artículos académicos, blogs e informes de la industria, concluyendo que uno de los aspectos más importantes es diseñar correctamente los límites de cada microservicio desde el inicio, utilizando conceptos como *Bounded Context*. También encontraron que las dependencias excesivas entre servicios dificultan su evolución. Además, destacan la importancia de prácticas DevOps como CI/CD y monitoreo continuo, así como la necesidad de considerar la seguridad desde las primeras etapas del diseño.

Nota personal:

Esta fuente me ayudó a comprender que el éxito de una arquitectura de microservicios depende mucho de un buen diseño inicial y de mantener bajas las dependencias entre servicios. También entendí la importancia de las prácticas DevOps y de considerar la seguridad desde etapas tempranas del desarrollo.

Ficha Bibliográfica 5

Referencia:

Vatter, J., Mayer, R., & Jacobsen, H. A. (2024). *The Evolution of Distributed Systems for Graph Neural Networks and Their Origin in Graph Processing and Deep Learning: A Survey*. ACM Computing Surveys, 56(1).

Enlace:

<https://doi.org/10.1145/3597428>

Resumen:

El artículo analiza la evolución de los sistemas distribuidos para Graph Neural Networks (GNNs), explicando cómo combinan técnicas de procesamiento de grafos y aprendizaje profundo. El trabajo destaca la necesidad de crear soluciones escalables debido al crecimiento de los grafos en aplicaciones como redes sociales, sistemas de recomendación y biología. Además, compara diferentes métodos de particionamiento, muestreo, sincronización y paralelismo utilizados para entrenar modelos de manera eficiente en múltiples máquinas. También revisa herramientas y frameworks especializados para GNNs, mostrando sus ventajas y limitaciones. En general, el artículo sirve como una guía para comprender los retos y avances actuales en el entrenamiento distribuido de GNNs.

Nota personal:

Esta fuente me ayudó a entender cómo las Graph Neural Networks combinan procesamiento de grafos y aprendizaje profundo para trabajar con grandes cantidades de datos. Además, comprendí mejor los desafíos de escalabilidad, comunicación y entrenamiento distribuido en sistemas modernos.